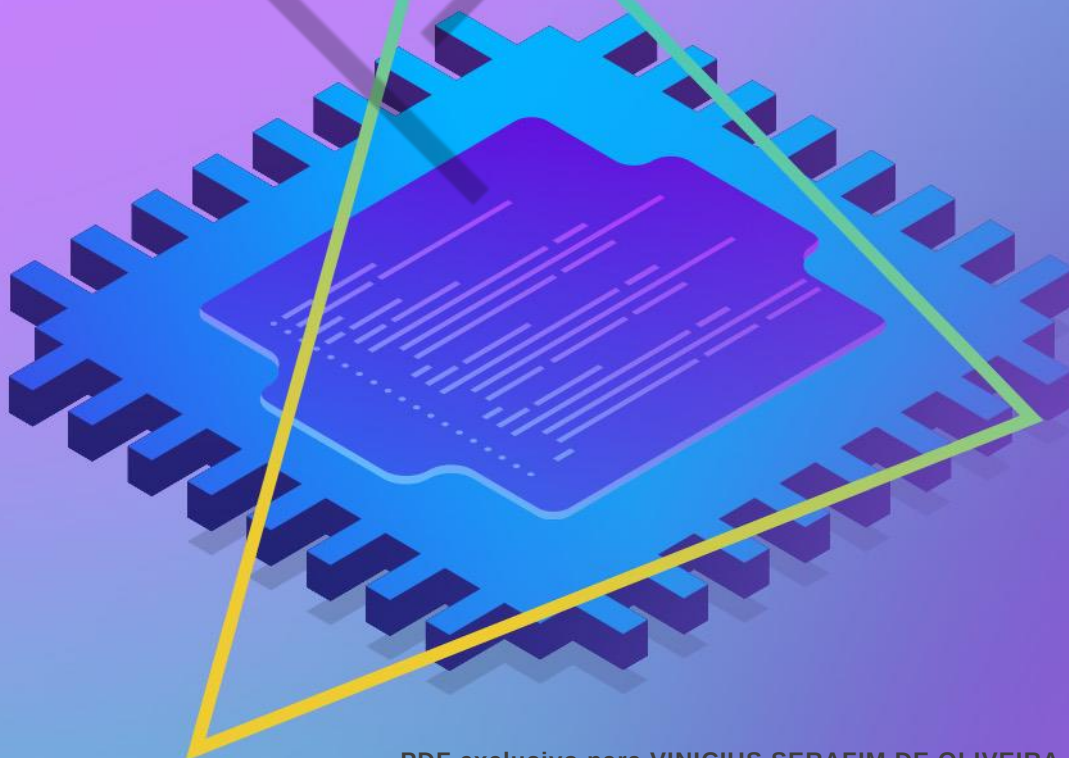


TECH FORCE: GALACTIC
ARCHITECTURE

GERENCIAMENTO DE MEMÓRIA



3

LISTA DE FIGURAS

Figura 1 - Exemplo de esquema de memória utilizando <i>Single-User Contiguous</i>	9
Figura 2 - Exemplo de esquema de memória utilizando o <i>Fixed Partition</i>	11
Figura 3 - Fragmentação interna.....	12
Figura 4 - Exemplo de esquema de memória utilizando o <i>Dynamic Partition</i>	13
Figura 5 - Troca de processos e fragmentação externa.....	14
Figura 6 - Desfragmentação de memória.....	15
Figura 7 - <i>First fit example</i>	16
Figura 8 - Alocando 25 K de memória usando o algoritmo de First Fit.....	16
Figura 9 - Alocando 100 K de memória usando o algoritmo de First Fit.....	16
Figura 10 - Alocando 70 K de memória usando o algoritmo de Best Fit	17
Figura 11 - Alocação de 200Kb de memória usando o algoritmo de Best Fit.....	17
Figura 12 - Exemplo de Memória Virtual (Windows Internals 7th edition - part 1).....	21
Figura 13 - Layouts de espaço de endereço virtual em sistemas 32-bit (2 GB à esquerda, 3 GB à direita)	22
Figura 14 - Consumo de memória no gerenciador de tarefas	24
Figura 15 - Exemplo de mapeamento de memória em sistemas antigos e sistemas modernos	26
Figura 16 - Alocando memória com a função VirtualAlloc	28
Figura 17 - Usando a função WriteProcessMemory	29
Figura 18 - Alocando espaço de memória no endereço 0x1090000	30
Figura 19 - Escrevendo no espaço de memória alocado anteriormente	30
Figura 20 - Liberação do espaço de memória.	31
Figura 21 - Utilizando a função mmap no Linux para alocar memória	32
Figura 22 - Exemplo de escrita de memória em sistema GNU/Linux	34
Figura 23 - Alocando espaço de memória no endereço 0xF7F82000	34
Figura 24 - Escrevendo no espaço de memória alocado anteriormente	34
Figura 25 - Liberação de espaço de memória	35
Figura 26 - Uso de memória do nosso sistema-alvo	36
Figura 27 - Execução do commando de <i>fork bomb</i> com o PID de 1731	37
Figura 28 - Recursos ao extremo	37
Figura 29 - Lixo deixado na memória	39
Figura 30 - Alocação de memória	40
Figura 31 - Liberação de Memória	40

LISTA DE TABELAS

Tabela 1 – Estado e descrição de <i>free</i> , <i>reserved</i> e <i>committed</i>	23
---	----

EMANIP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplos de alocação de liberação de memória em C/C++	18
Código-fonte 2 – Um simples exemplo de alocação de memória em sistemas Windows.....	27
Código-fonte 3 – Um simples exemplo de alocação de memória em sistemas GNU/Linux.....	31
Código-fonte 4 – Exemplo de programa que não faz a liberação de memória.....	39
Código-fonte 5 – Exemplo de código realizando o tratamento adequando de memória	40

SUMÁRIO

GERENCIAMENTO DE MEMÓRIA.....	6
1 INTRODUÇÃO	6
2 O QUE É O GERENCIAMENTO DE MEMÓRIA?	7
2.1 Técnicas de gerenciamento de memória	8
2.1.1 <i>Single-User Contiguos</i>	9
2.1.2 <i>Fixed Partitions</i>	10
2.1.3 <i>Dynamic Partitions</i>	12
2.1.4 <i>Memory Defragmentation</i>	14
2.1.5 <i>First & Best Fit Allocation</i>	15
2.1.5.1 <i>First Fit Algorithm</i>	15
2.1.5.2 <i>Best Fit Algorithm</i>	17
2.1.6 <i>Memory Deallocation</i>	18
2.2 <i>Virtual Memory & Paging</i>	20
2.2.1 <i>Paging</i>	23
2.2.2 <i>Swapping</i>	23
2.2.3 <i>Mapeamento de memória</i>	25
2.2.3.1 <i>Endereço virtual (VA)</i>	26
2.2.3.2 <i>Endereço virtual relativo (RVA)</i>	26
3 EXEMPLOS DE ALOCAÇÃO DE MEMÓRIA EM SISTEMAS WINDOWS E GNU/LINUX.....	27
3.1 <i>Alocação de Memória em Sistemas Windows</i>	27
3.2 <i>Alocação de memória em sistema GNU/Linux</i>	31
4 ATAQUES EM MEMÓRIA.....	35
4.1 <i>Fork Bomb</i>	35
5 PROTEÇÕES DE MEMÓRIA.....	38
5.1 <i>Garbage Collection</i>	38
5.2 <i>Data Execution Prevention (DEP)</i>	40
5.3 <i>Address Space Layout Randomization (ASLR)</i>	41
CONCLUSÃO.....	41
REFERÊNCIAS.....	42

GERENCIAMENTO DE MEMÓRIA

1 INTRODUÇÃO

Em um mundo cada vez mais digitalizado, no qual a quantidade e a complexidades dos dados manipulados pelos computadores crescem exponencialmente, o gerenciamento de memória torna-se essencial para garantir um bom desempenho e a confiabilidade dos sistemas computacionais.

Neste capítulo, exploraremos em detalhes os conceitos e as técnicas essenciais para o gerenciamento de memória. Compreenderemos os diferentes esquemas de memória, desde os sistemas mais antigos até as soluções mais avançadas atualmente empregadas em sistemas operacionais modernos.

Inicialmente, iremos abordar o que é o gerenciamento de memória, esclarecendo seu propósito e sua importância para o funcionamento adequado dos computadores. Em seguida, exploraremos alguns esquemas de gerenciamento de memória desenvolvidos ao longo do tempo, como: *Single-User Contiguous* (único usuário contíguo), *Fixed Partitions* (partições fixas), *Dynamic Partitions* (partições dinâmicas) e os algoritmos de alocação de memória *First Fit* (primeiro encaixe) e *Best Fit* (melhor encaixe). Além disso, discutiremos o processo de deslocação (ou liberação) de memória.

Posteriormente, iremos focar no conceito de *Memory Mapping* (mapeamento de memória), *Virtual Memory* (memória virtual) e *Paging* (paginação), conceitos que revolucionaram o gerenciamento de memória, permitindo o uso eficiente da capacidade de armazenamento disponível. Veremos como a paginação permite que os programas acessem mais espaço de memória do que está fisicamente disponível no sistema.

Em seguida, iremos aprender como é feita a alocação de memória em sistema Windows e em sistemas GNU/Linux. Por fim discutiremos algumas vulnerabilidades de memória e as soluções empregadas para solucionar tais problemas.

Com este capítulo, você terá uma compreensão bastante abrangente sobre o gerenciamento de memória, suas técnicas, a sua evolução ao longo do tempo, sua importância para a segurança e para o desempenho dos sistemas operacionais.

2 O QUE É O GERENCIAMENTO DE MEMÓRIA?

O gerenciamento de memória é um conjunto de técnicas e algoritmos utilizados para controlar a alocação, o uso e a liberação da memória em sistemas operacionais. Como visto anteriormente, a memória principal (também conhecida como memória RAM) é um recurso fundamental para o funcionamento de um computador, vimos que ela é responsável pelo armazenamento de programas em execução (ou processos), dos dados necessários para suas operações e os seus resultados intermediários de processamentos.

Dessa forma o gerenciamento de memória precisa performar algumas ações para possibilitar com que a *Central Processing Unit* (CPU, ou em português, unidade central de processamento - UCP) obtenha acesso aos dados dos programas para assim poder executá-los. Abaixo listamos uma breve explicação de ações relevantes que são performadas pelo gerenciador de memória:

- **Validação da requisição:** O gerenciador de memória valida a requisição de alocação de memória feita por uma aplicação e se há espaço suficiente na memória para atender adequadamente os recursos dessa aplicação.
- **Alocação:** Desempenha uma busca por partições livres que satisfaçam uma determinada requisição de memória, alocando-as com os recursos necessários.
- **Deslocação:** O deslocamento de memória realiza a limpeza de processos ou tarefas encerradas ou ociosas.
- **Desfragmentação:** Executa o rearranjo da memória para estabelecer partições de memória livre de forma contígua.

Um bom gerenciamento de memória possui um grande impacto no desempenho e na segurança dos sistemas operacionais.

Quando relacionado a segurança de memória, o gerenciamento de memória possui um papel crucial para a proteção contra vulnerabilidades e ataques maliciosos, como por exemplo o *Buffer Overflow*, *Memory Leak* (ou vazamento de memória), *Denial of Service* (DoS).

Quando performed uma alocação inadequada de memória isso pode levar a problemas como a fragmentação.

A fragmentação ocorre quando pequenos espaços de memória são alocados para diferentes tarefas de forma não contígua.

Os resultados gerados por conta dessa forma inadequada de alocação fazem com que, mesmo ainda havendo espaços disponíveis de memória esses se tornam inutilizáveis, pois, como esses espaços estão divididos em tamanhos menores se torna difícil satisfazer as exigências de uma determinada aplicação. Isso pode causar o desperdício de recursos e a diminuição da eficiência do sistema.

Por outro lado, um gerenciamento eficiente de memória permite que os processos sejam alocados e executados de forma otimizada, garantindo o máximo aproveitamento dos recursos disponíveis assim melhorando o desempenho geral do sistema.

Como podemos ver, o gerenciamento de memória desempenha um papel importantíssimo no desenvolvimento de software. Trazendo benefícios que contribui para a criação de softwares confiáveis, seguros, e eficientes, melhorando a experiência do usuário e garantindo a qualidade das aplicações.

2.1 Técnicas de gerenciamento de memória

Agora que entendemos o quão importante e como o gerenciamento de memória pode beneficiar no desenvolvimento de software e na sua qualidade, vamos dar uma olhada em diferentes técnicas e algoritmos de alocação de memória que foram utilizados ao longo do tempo.

Diversas técnicas para o gerenciamento de memória foram desenvolvidas ao longo do tempo, sempre buscando melhorar o desempenho, a segurança e a utilização da memória de forma efetiva nos sistemas operacionais. Veremos como cada uma dessas técnicas possuíram um papel significativa que complementou com a evolução para a próxima.

Neste tópico, iremos explorar os diferentes esquemas de gerenciamento de memória e sua evolução ao longo do tempo. Ao explorar esses esquemas, teremos uma boa compreensão das estratégias adotadas para otimizar a alocação e o uso da memória de forma eficaz.

2.1.1 *Single-User Contiguos*

Esse é um esquema de gerenciamento de memória que foi amplamente utilizado nos primórdios dos sistemas operacionais. Com essa abordagem, a memória disponível no sistema era dividida em uma única partição contígua, que era alocada exclusivamente para um processo.

Cada programa que era executado recebia uma partição contígua de memória, que era alocada com base em suas necessidades durante o carregamento. A área que era alocada a esse programa permanecia exclusivamente reservada a ele até que ele fosse encerrado ou liberado da memória de forma explícita.

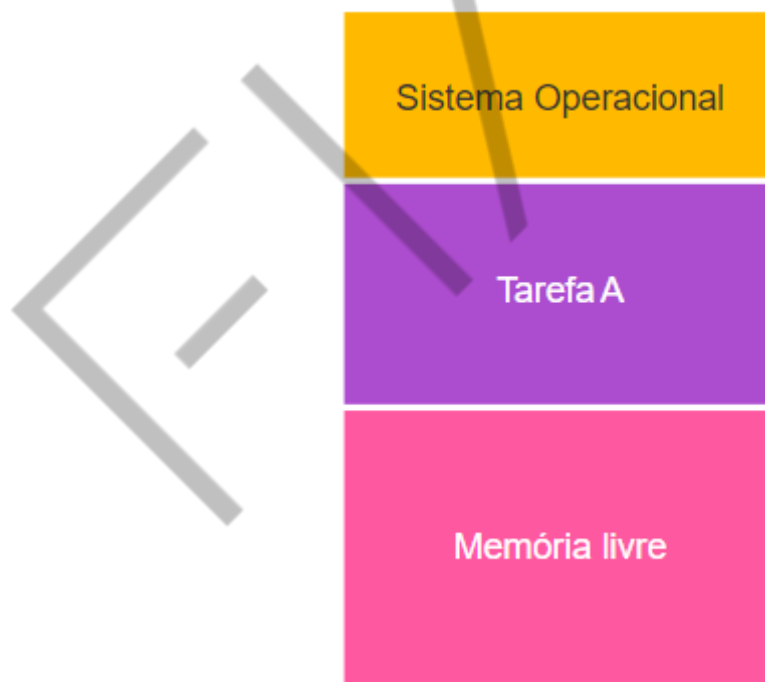


Figura 1 - Exemplo de esquema de memória utilizando *Single-User Contiguous*
Fonte: Elaborada pelo autor (2023)

Uma de suas vantagens era sua simplicidade, pois a alocação realizara por esse esquema era direta e não necessitava de muitos recursos de hardware ou algoritmos complexos para realizar seu objetivo.

Porém, suas desvantagens eram maiores. Esse esquema possui a desvantagem de que, **apenas um programa era permitido ser executado por vez**, mesmo havendo mais espaço de memória disponível, assim acabou sendo considerado uma opção muito lenta para as demandas que foram surgindo. Outro problema era se caso um programa requisitasse uma alocação maior do que o espaço de memória disponível no sistema, esse programa simplesmente não executava.

Com a evolução dos sistemas operacionais e com o aumento das demandas de memória das aplicações, novos esquemas eram necessários. Como vimos esquemas como o *Single-User Contiguous* já não estavam satisfazendo as necessidades da época. Para solucionar esse problema um novo esquema de memória foi desenvolvido. Esse sendo o *Fixed Partitions*.

2.1.2 Fixed Partitions

Esse esquema de memória foi desenvolvido para solucionar alguns dos desafios anteriores existentes no esquema de *Single-User Contiguous*, o qual não permitia *Multiprogramming* (múltiplos programas). Com esse novo esquema de gerenciamento de memória, a memória física do sistema era **dividida em partições fixas de tamanhos predefinidos**. Cada uma das partições era designada para um processo específico e alocada exclusivamente para ele durante seu tempo de execução. Essas partições eram definidas durante a inicialização do sistema operacional e permaneciam inalteradas durante toda a sua operabilidade.

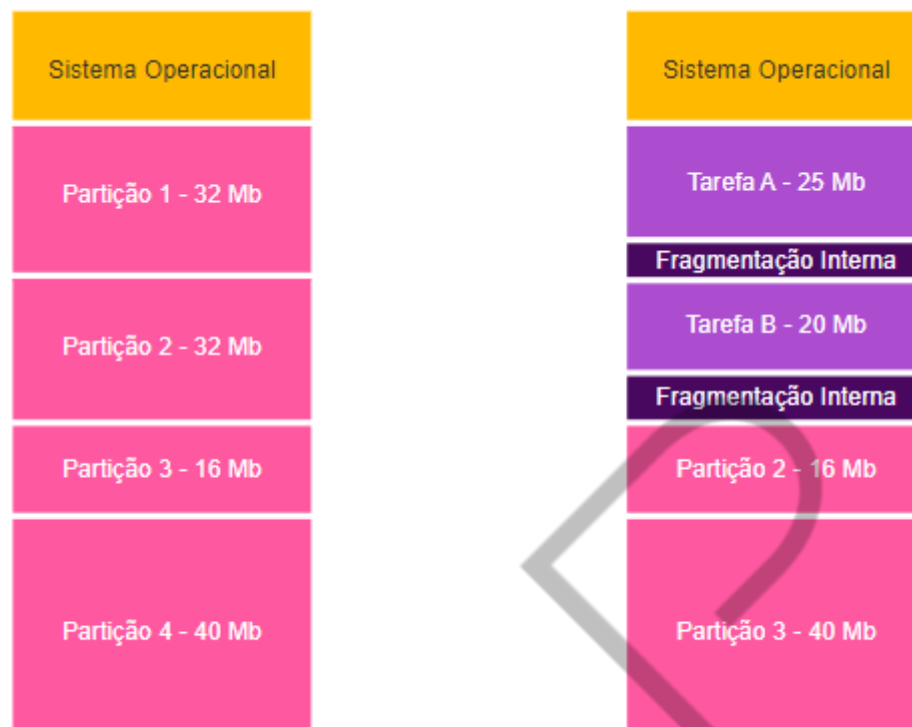


Figura 2 - Exemplo de esquema de memória utilizando o *Fixed Partition*
Fonte: Elaborada pelo autor (2023)

Como podemos ver na figura “Exemplo de esquema de memória utilizando o *Fixed Partition*”, à esquerda, vemos o tamanho que foi alocado para cada partição, já a direita vemos os processos alocados em uma partição que satisfaça as necessidades de tal processo, para exemplo didático estamos utilizando um exemplo de memória com o tamanho de 128 Mb.

Porém, conforme novas necessidades foram surgindo, esse esquema de memória também começou a apresentar algumas de suas desvantagens, como:

1. As partições de memória precisavam ser particionadas cada vez que um processo fosse maior que o tamanho disponível de partições alocadas anteriormente. Sem falar que para isso era necessário reinicializar o sistema.
2. Outros problemas era que o sistema desperdiçava muito espaço de memória dentro das partições. Isso ficou conhecido como *Fragmentação interna*, esse problema se refere quando um processo é alocado em uma partição, desde que a partição fosse maior que o que foi requisitado pelo processo. Assim deixando espaços vazios dentro das partições, como podemos ver na figura abaixo.

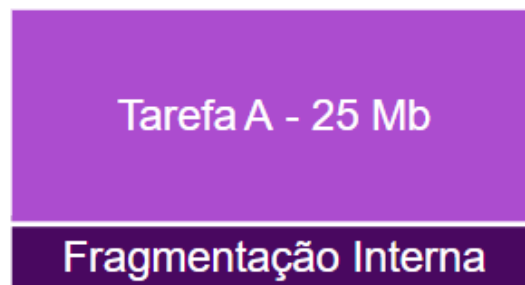


Figura 3 - Fragmentação interna
Fonte: Elaborada pelo autor (2023)

Algumas das vantagens que foram oferecidas por esse esquema de memória na época, foram:

- **Fácil implementação:** Um algoritmo simples que implementava um tamanho fixo de partições.
- **Baixa sobrecarga:** Esse esquema não necessitava de muito recurso para sua implementação, o que fazia ideal para sistema com recursos limitados.
- **Previsível:** Garantia a previsibilidade da quantidade de memória para cada processo.

Porém, o maior problema com esse esquema de memória era ter que reinicializar o sistema toda vez que era necessário alterar uma partição. Dessa forma causando a indisponibilidade de um serviço. Para isso era necessária uma forma de alocação dinâmica de memória.

2.1.3 *Dynamic Partitions*

Como visto anteriormente, o esquema de *Fixed Partition* possuía o problema de fragmentação interna quando um processo era alocado em um espaço predefinido de memória. Para solucionar esse problema, foi desenvolvido o esquema de Partições Dinâmica. Esse esquema operava da seguinte forma:

1. Espaços eram alocados de forma contígua com base na necessidade de cada processo.
2. Como os sistemas daquela época já suportavam múltiplos programas, diversos processos eram permitidos ser carregados para memória para serem executados.

Como podemos ver na figura “Exemplo de esquema de memória utilizando o *Dynamic Partition*”, cada alocação era alocada com base na necessidade de cada processo.



Figura 4 - Exemplo de esquema de memória utilizando o *Dynamic Partition*
Fonte: Elaborada pelo autor (2023)

Como podemos ver na Figura 4, toda a memória foi alocada para três tarefas (Tarefa A, B e C). Quando esses processos são removidos ou trocados, um novo problema apareceu com esse esquema de memória, e esse foi a **fragmentação externa**.

Com podemos ver na Figura 5, alguns espaços foram deixados quando a tarefa B é trocada pela D e quando a tarefa A é trocada pelo processo E.

A fragmentação externa de memória é o desperdício de memória, mas essa ocorre fora da partição.



Figura 5 - Troca de processos e fragmentação externa
Fonte: Elaborada pelo autor (2023)

Caso outro processo menor for trocado após a conclusão de suas tarefas, mais fragmentação externa serão criadas. Podendo assim chegar no ponto em que as partições existentes não poderão suportar novas tarefas. Para solucionar esses problemas, a **desfragmentação** de memória foi criada.

2.1.4 Memory Defragmentation

A solução criada para solucionar os problemas existentes com o esquema de partições dinâmicas foi a desfragmentação (também conhecida como Compactação de Memória). A compactação é uma maneira pela qual os sistemas operacionais selecionam todos os espaços vazios, agregando os espaços de memórias não utilizados em blocos contíguos, conforme representando na Figura 6.

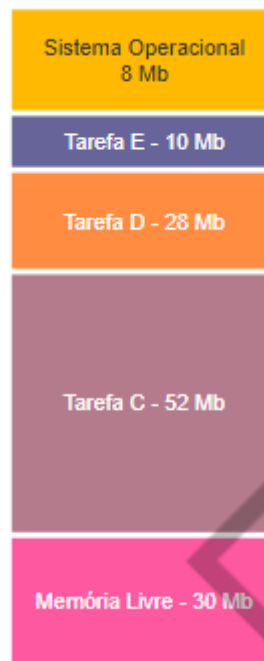


Figura 6 - Desfragmentação de memória
Fonte: Elaborada pelo autor (2023)

2.1.5 First & Best Fit Allocation

Com a evolução dos esquemas de memória, novas técnicas/algoritmos foram desenvolvidas para melhorar o desempenho e a qualidade durante a alocação de memória.

Existem diversos algoritmos utilizados para realizar alocação de memória, como *First Fit*, *Next Fit*, *Best Fit*, *Worst Fit*, *Quick Fit*. Porém só iremos focar em dois desses, sendo eles o *First* e o *Best Fit*.

2.1.5.1 First Fit Algorithm

Na técnica de First Fit, o gerenciador de memória procura a primeira partição de memória que esteja livre e que seja grande o suficiente para acomodar o processo a ser alocado. Ele procura na lista de partições de memória a partir do início e aloca o processo na primeira partição que satisfaz os requisitos do tamanho.

Por exemplo, imagine que temos as seguintes partições, e precisamos alocar os seguintes valores: **25 K**, **100 K**.

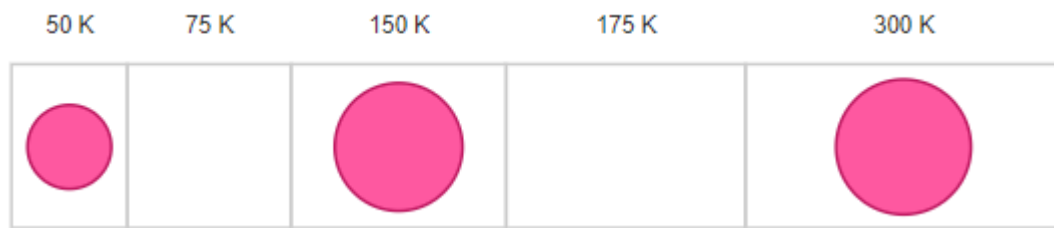


Figura 7 - First fit example
Fonte: Elaborada pelo autor (2023)

Então como dito antes, primeiro o algoritmo irá realizar um escaneamento em busca da primeira partição que seja grande o suficiente para alocar o tamanho requisitado pela aplicação.

Requisição de 25K: O primeiro espaço que satisfaz a requisição de 25 K é a segunda partição de 75 K, consequentemente é subtraído 25 K de 75 K sobrando 50 K.

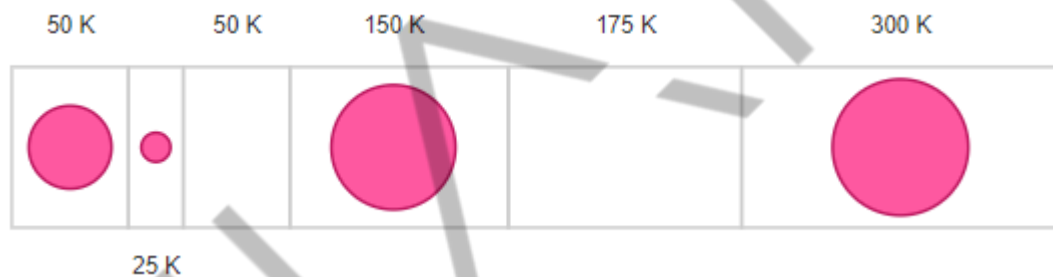


Figura 8 - Alocando 25 K de memória usando o algoritmo de First Fit
Fonte: Elaborada pelo autor (2023)

Requisição de 100 K: Já a segunda requisição pede um espaço de 100 K, então novamente é realizado o escaneamento para buscar um espaço de memória. Nesse caso será a quinta partição com 175 K a ser utilizada.



Figura 9 - Alocando 100 K de memória usando o algoritmo de First Fit
Fonte: Elaborada pelo autor (2023)

2.5.1.2 Best Fit Algorithm

Na técnica de *Best Fit*, o gerenciador de memória procura por uma partição disponível que tenha um tamanho mais próximo do tamanho solicitado pela aplicação. Esse algoritmo realiza um escaneamento na lista de partições e seleciona a menor partição, porém que satisfaça o tamanho da aplicação.

Agora vamos ver outro exemplo de alocação de memória com outros valores sendo esses, **70 K** e **200 K**, agora utilizando os algoritmos de Best Fit.

Requisição de 70 K: Neste caso para satisfazer uma requisição de 70 K, a partição selecionada por esse algoritmo foi a segunda que continha um espaço de 75 K.

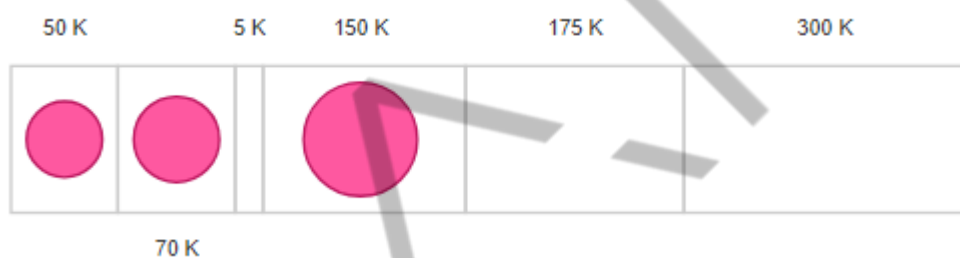


Figura 10 - Alocando 70 K de memória usando o algoritmo de Best Fit
Fonte: Elaborada pelo autor (2023)

Requisição de 200 K: Já para alocar 200 K para a segunda aplicação, é escaneado novamente as partições com espaço livres. Nesse exemplo vemos que o algoritmo de *Best Fit* selecionou a partição de 300 K.

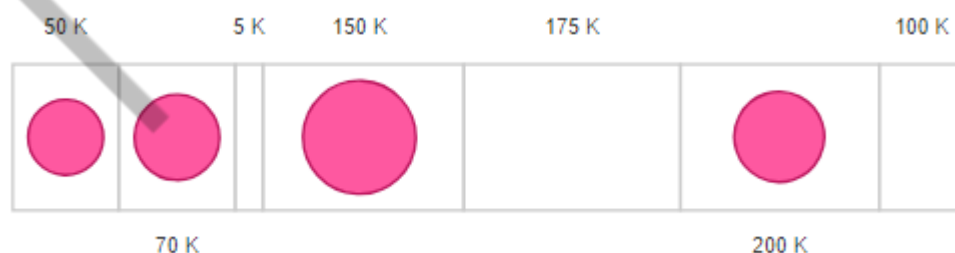


Figura 11 - Alocação de 200Kb de memória usando o algoritmo de Best Fit
Fonte: Elaborada pelo autor (2023)

Ambas as técnicas possuem suas vantagens e desvantagens. **O First Fit é mais rápido, mas pode levar à fragmentação interna. Já o Best Firt é mais eficiente em temos de uso de memória, mas pode ser mais lento.** A escolha entre

as duas técnicas depende das características do sistema e dos requisitos de alocação de memória.

2.1.6 Memory Deallocation

O deslocamento de memória, também conhecido como liberação, refere-se ao processo de liberar um espaço de memória alocada anteriormente para o uso de um programa. É de extrema importância garantir que essa memória seja liberada quando não for mais ser necessário para uso. A falta de liberação de memória pode levar a problemas como *Memory Leak* (vazamento de memória), veremos isso mais para frente.

A liberação de memória é particularmente importante em linguagens de programação que não possuem *Garbage Collection* (coleta de lixo), como C e C++. Nessas linguagens, responsabilidade de liberar a memória alocada recai sobre o desenvolvedor.

Existem diferentes métodos para realizar a liberação de memória, isso depende da forma com que ela foi alocada. Abaixo podemos ver alguns exemplos na linguagem C/C++.

```
// 1 - Usando as funções malloc() e free()
PVOID address = malloc(100);
free(address);

// 2 - Usando as funções HeapAlloc() e HeapFree()
PVOID pAddress = HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, 100);
HeapFree(GetProcessHeap(), HEAP_ZERO_MEMORY, pAddress);

// 3 - Usando as funções LocalAlloc() e LocalFree()
PVOID pAddress = LocalAlloc(LPTR, 100);
LocalFree(pAddress);

// 4 - Usando as funções VirtualAlloc() e VirtualFree()
LPVOID pAddress = VirtualAlloc(
    NULL,
    strlen(string),
    (MEM_COMMIT | MEM_RESERVE),
    PAGE_READWRITE
);
VirtualFree(pAddress, 0, MEM_RELEASE);
```

Código-fonte 1 – Exemplos de alocação de liberação de memória em C/C++

Fonte: Elaborado pelo autor (2023)

Em resumo, as diferenças principais estão relacionadas ao tipo de alocação (heap local, heap do processo ou espaço de endereçamento virtual) e ao modo como

a memória é liberada (marcada como disponível para uso futuro ou descomprometida). A escolha da função apropriada depende das necessidades específicas do programa e do tipo de memória que precisa ser alocada.

1. `malloc()` e `free()`:

- São adequadas para alocação de memória dinâmica quando você precisa controlar o tamanho exato do bloco de memória.
- São úteis para alocar e liberar memória de forma genérica em programas que não exigem recursos de alocação especializados.

2. `HeapAlloc()` e `HeapFree()`:

- São úteis quando você precisa alocar memória do heap do processo, que é uma área gerenciada pelo sistema operacional.
- Podem ser úteis quando você está trabalhando com bibliotecas ou interfaces de programação de aplicações (*Application Programming Interface* – APIs) específicas do Windows que exigem o uso dessas funções.

3. `LocalAlloc()` e `LocalFree()`:

- São úteis quando você precisa alocar memória no heap local do processo.
- Podem ser úteis quando você precisa alocar memória para recursos específicos do Windows ou quando trabalha com código legado que faz uso dessas funções.

4. `VirtualAlloc()` e `VirtualFree()`:

- Essas funções são usadas quando você precisa trabalhar com o espaço de endereçamento virtual do processo.
- São úteis quando você precisa reservar uma região de endereços para mapeamento de arquivos, alocação de memória compartilhada ou criação de regiões de memória com proteções especiais.
- Podem ser úteis quando você está trabalhando com baixo nível de programação, como manipulação direta de páginas de memória.

2.2 Virtual Memory & Paging

Agora que aprendemos um pouco sobre a evolução de diferentes esquemas de memória, vamos focar nossa atenção ao esquema que utilizamos nos dias de hoje, tendo como foco mais voltado para o sistema Windows.

O conceito de memória virtual é fundamental nos sistemas operacionais modernos. O sistema Windows realiza a implementação da memória virtual baseado em endereços simples (ou linear). Esse esquema de memória possibilita uma “ilusão” para cada processo como se cada um possuísse seu próprio grande espaço privado de memória.

A ideia com esse esquema de memória diferente dos outros apresentados anteriormente, é que em vez de permitir que a aplicação acesse diretamente a memória física do computador, o processador em conjunto do sistema operacional, cria uma camada “invisível” entre a aplicação e a memória física do dispositivo. Para cada acesso à memória, é necessário que o processador realize uma consulta a uma tabela chamada *Page Table* (tabela de página).

Essa tabela tem como objetivo informar ao processador qual endereço de memória RAM realmente utilizar.

Como veremos a seguir, essa tabela permite o mapeamento entre os endereços virtuais para os endereços físicos da memória principal.

O uso desse esquema de memória permite com que o sistema operacional execute programas que são muitas vezes maiores do que o tamanho da memória física disponível no computador. Permitindo dessa forma estender a capacidade de memória RAM usando uma combinação de memória principal e memória secundária (ou disco rígido). Veremos mais sobre isso um pouco mais a frente. A figura abaixo ilustra muito bem esse esquema de memória.

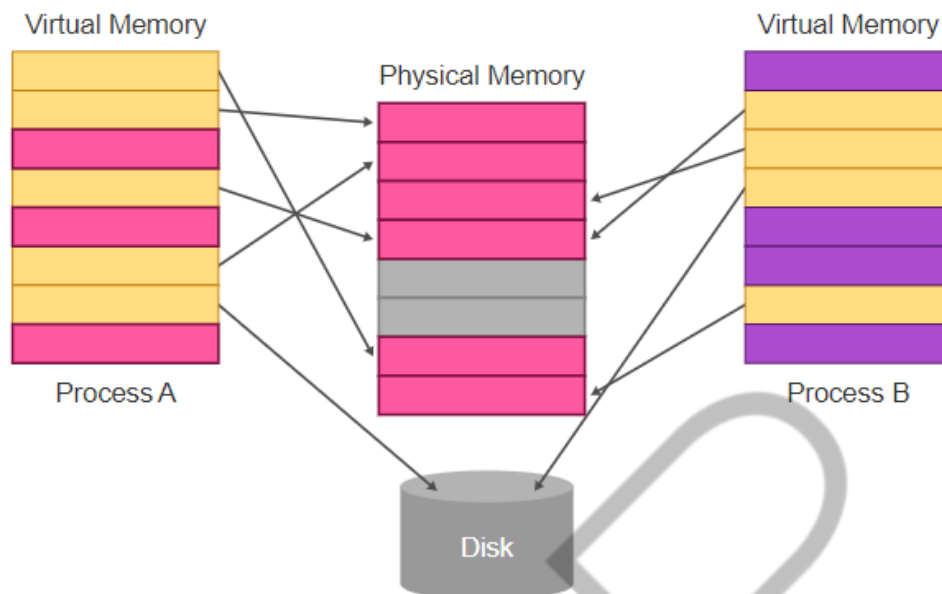


Figura 12 - Exemplo de Memória Virtual (Windows Internals 7th edition - part 1)
Fonte: Google Imagens (2023)

A Paginação de memória é um mecanismo essencial usado na implementação de memória virtual. Seu objetivo é dividir a memória principal em pedaços chamados *Pages* (páginas). Essas páginas são blocos de memória de tamanho fixo. **Por padrão cada página possui um tamanho de 4 KB.**

Em teoria, isso significa que um sistema de 32 bits pode endereçar no máximo 2^{32} endereços de memória. Como cada endereço de memória normalmente representa um byte, o tamanho máximo teórico da memória endereçável é aproximadamente 4 GB (gigabytes). De acordo com Microsoft (2023) "(Cada processo no Microsoft Windows de 32 bits tem seu próprio espaço de endereço virtual que permite endereçar até 4 gigabytes de memória. Cada processo no Windows de 64 bits tem um espaço de endereço virtual de 8 terabytes.)".

Com isso por padrão, o Windows de 32 bits divide o espaço de endereço virtual de 4 GB pela metade para os processos do usuário e outro para o sistema operacional em si. A metade inferior do espaço de endereços entre (0x00000000 a 0x7FFFFFFF) é alocada aos processos para seu armazenamento privado.

Isso significa que cada processo possui um espaço de endereço virtual de 2 GB disponível para o seu próprio uso.

Já a metade superior do espaço de endereços entre (0x80000000 a 0xFFFFFFFF) são reservados para a utilização da memória do sistema operacional.

Isso inclui o kernel, drivers de dispositivos e outros componentes necessários para o funcionamento adequado do sistema operacional. Essa divisão garante que os processos não possam acessar ou sobrescrever a memória crítica do sistema operacional, aprimorando a estabilidade e a segurança do sistema.

Aqui vale ressaltar que é possível configurar o sistema Windows para que ele forneça aos processos a capacidade de usar até 3 GB de espaço de endereçamento privado, deixando 1 GB para o sistema operacional.

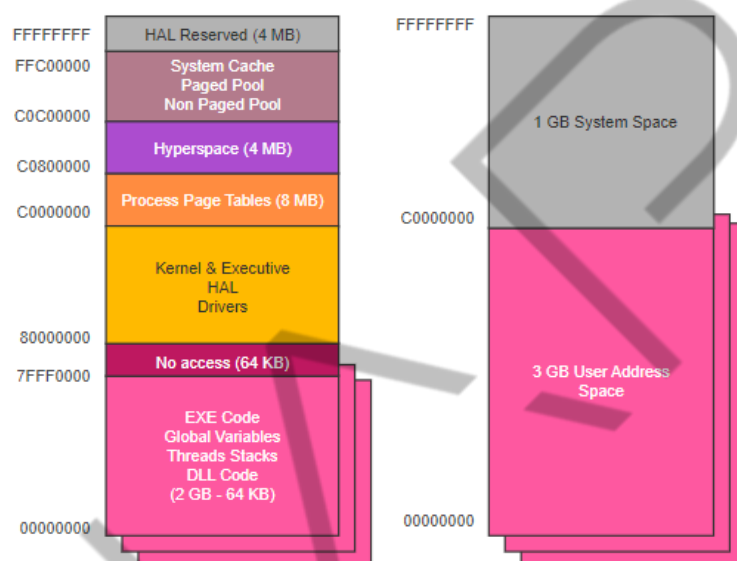


Figura 13 - Layouts de espaço de endereço virtual em sistemas 32-bit (2 GB à esquerda, 3 GB à direita)

Fonte: Google Imagens (2023)

Já o espaço de endereçamento de sistemas 64-bits como mencionado antes permite que uma grande quantidade de memória seja endereçada. O tamanho máximo em teoria da memória endereçável em um sistema de 64 bits é de 2^{64} bytes, que é um valor enorme que excede as capacidades das tecnologias de memória física atuais. As limitações atuais permitem apenas 48 linhas de endereço, limitando o espaço de endereço possível a 256 TB (2 elevado a 48ª potência). O espaço de endereço é dividido ao meio, onde os 128 TB inferiores estão disponíveis como processos privados do usuário e os 128 TB superiores são espaço do sistema.

2.2.1 Paging

A página que reside no espaço de endereço virtual de um processo pode estar em um dos 3 estados listados na tabela abaixo.

Estado	Descrição
Free (Livre)	A página não está nem no estado <i>Committed</i> (confirmada) ou <i>Reserved</i> (Reservada). Ou seja, a página não está acessível ao processo. Mas essa página está disponível para ser reservada, confirmada ou os dois ao mesmo tempo. A tentativa de ler ou escrever em uma página no estado <i>Free</i> pode resultar em uma exceção de violação de acesso.
Reserved (Reservada)	A página foi reservada para uso futuro. O intervalo de endereço não pode ser usado por outras funções de alocação. Ou seja, essa página não está acessível e não possui armazenamento físico associado a ela.
Committed (Confirmada)	Uma página em estado <i>committed</i> (ou confirmada) são acessíveis e são controladas por uma constante de proteção de memória. O sistema inicializa e carrega cada página confirmada na memória principal apenas durante a primeira tentativa de ler ou gravar nessa página. Quando o processo termina, o sistema libera o armazenamento das páginas confirmadas.

Tabela 1 – Estado e descrição de *free*, *reserved* e *committed*

Fonte: Elaborada pelo autor (2016)

Algumas proteções de memória são:

- **PAGE_NOACCESS** – Desativa todo o acesso à região confirmada de páginas.
- **PAGE_EXECUTE_READWRITE** – Habilita leitura, escrita e execução.
- **PAGE_READONLY** – Habilita o acesso somente a leitura e escrita da região confirmada de páginas.

Para realizar a alteração de uma página de memória no sistema Windows, você precisa utilizar a função **VirtualProtect()**.

2.2.2 Swapping

Como mencionado antes, o esquema de memória virtual possibilita a execução de um programa maior do que o tamanho disponível de memória física no computador. Pois, partes dos processos que não estão em execução são carregados para a memória secundário permitindo assim que novos processos sejam executados. Isso é conhecido como *Swapping* ou troca de processo.

Tanto em sistemas Windows com em Linux, quando é ligado o computador pode-se inicializar cerca de mais de 100 processos. Imagine uma aplicação sendo instalada no seu sistema Windows, por exemplo um jogo. Essa nova aplicação realizara muitas execuções de diversos comando, como a criação de diretórios onde será armazenado os arquivos necessários para execução do jogo, realizará também uma validação na internet para ver se não há alguma atualização recente que precisa ser feita. Apesar que hoje em dia muitos executáveis que realizam a instalação de programa grandes realizam o download dos arquivos necessários diretamente do site dos desenvolvedores. Mesmo assim, diversas ações serão desempenhadas no dispositivo. Enquanto isso, outros processos são executados em background (ou segundo plano), como por exemplo, validando se há novas mensagens em sua caixa de e-mail, recebendo e enviando tráfego de rede para renderizar um vídeo no Youtube, e muita mais outras coisas.

Assim que ligamos nosso computador de contém **8 GB** de memória RAM, vemos na imagem abaixo que cerca de **2.1 GB** estão sendo consumidos só pelos processos necessários para a execução do sistema operacional Windows. E tudo isso antes mesmo de executar nosso primeiro programa de usuário. Essa figura foi tirada de um sistema recém-instalado contendo apenas as aplicações e configurações padrão do Windows.

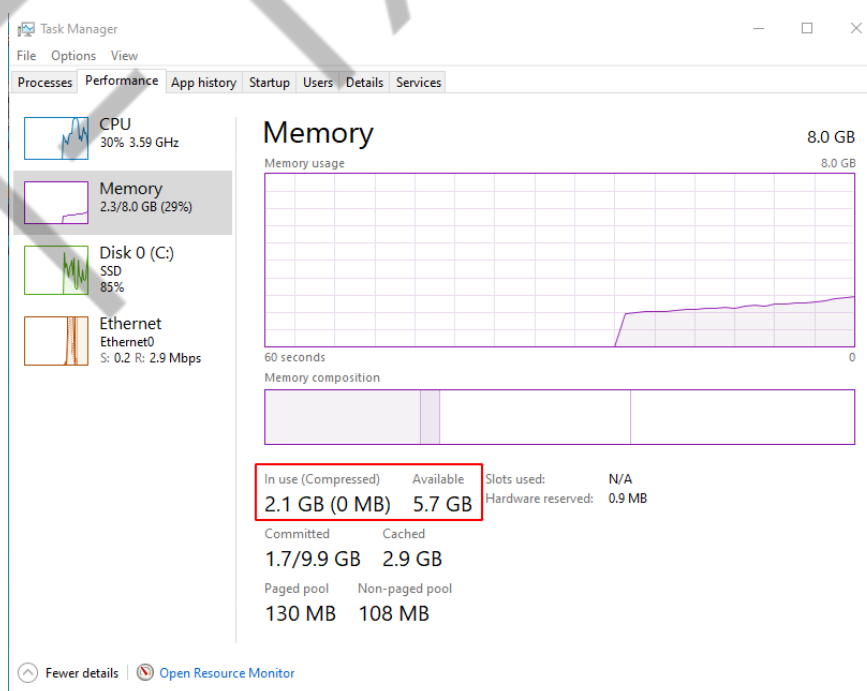


Figura 14 - Consumo de memória no gerenciador de tarefas
Fonte: Google Imagens (2023)

Em casos como esse onde o dispositivo possui pouco recurso de memória RAM, manter todos os processos na memória o tempo inteiro é algo desnecessário. Duas soluções para lidar com essa sobrecarga de memória foram criadas, como visto anteriormente a paginação e a memória virtual foi uma delas. A outra era a troca de processos que não estavam em execução.

O *Swapping* envolve a transferência dos dados entre a memória RAM para o disco rígido (ou outro dispositivo de armazenamento secundário) para dessa forma possibilitar a liberação de espaços na memória principal. Então quando um programa em execução não está sendo ativamente utilizada, o sistema operacional pode optar por mover partes desse programa da memória principal para o disco. Dessa forma, o espaço de memória é liberado para outros programas que precisam ser executados ou para dados que estão sendo usados mais ativamente.

Esse processo de troca de processos é transparente para os programas em execução, uma vez que o sistema operacional é responsável por gerenciar essa transferência de dados. No entanto, o uso do *Swapping* pode afetar o desempenho do sistema, uma vez que o acesso a dados no disco rígido é significativamente mais lento do que o acesso à memória RAM. Portanto, seu uso excessivo pode resultar em uma diminuição perceptível no desempenho geral do sistema. Normalmente se é utilizado em casos como o da figura “Exemplo de mapeamento de memória em sistemas antigos e sistemas modernos”.

2.2.3 Mapeamento de memória

Vimos anteriormente que a utilização da memória virtual realiza uma tradução de endereços virtuais para endereços físicos de memória. Isso é realizado pela unidade de gerenciamento de memória (*Memory Management Unit* - MMU), esse é um dispositivo de hardware que traduz endereços virtuais para endereços físicos. Geralmente é implementado como parte da CPU, mas também pode estar na forma de um circuito integrado separado.

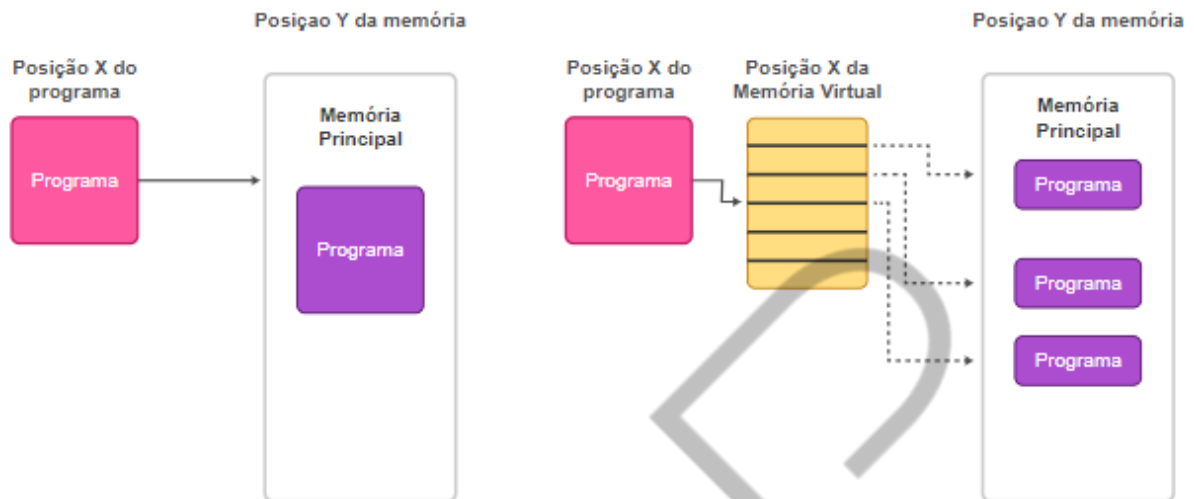
Alocação de memória em Sistemas antigos**Alocação de memória em Sistemas baseados em Memória Virtual**

Figura 15 - Exemplo de mapeamento de memória em sistemas antigos e sistemas modernos
Fonte: Google Imagens (2023)

2.2.3.1 Endereço virtual (VA)

O termo endereço virtual representa o endereço de memória atribuído a um determinado local na memória do sistema.

Quando um programa é executado, vimos que ele é carregado do disco para a memória RAM. Cada instrução de dado é referenciada por um endereço específico. No entanto, o endereço virtual não é o mesmo que o endereço físico real da memória. Em vez disso, é um endereço lógico atribuído pela MMU. Assim sendo utilizado como uma camada de abstração que permite ao sistema operacional mapear os endereços virtuais em endereços físicos reais na memória do computador.

2.2.3.2 Endereço virtual relativo (RVA)

O endereço virtual relativo (*Relative Virtual Address*) esse é apenas um deslocamento de onde a imagem do binário foi carregada na memória (a base da imagem). Dessa maneira, para traduzir um RVA em um endereço virtual absoluto, precisamos adicionar o valor do RVA ao valor da base da imagem.

Por exemplo, ao analisar um binário, você pode encontrar referência os endereços relativos (RVA) em vez de endereços absolutos. Essas referências são usadas para ajudar no processo de realocação do programa na memória.

O RVA é usado dessa forma para facilitar o carregamento do programa em memória em diferentes locais, pois assim permite que o programa seja realocado em sem precisar modificar diretamente os endereços absolutos em todo o código. Dessa forma, os endereços são expressos como deslocamento em relação a um ponto de referência, geralmente uma tabela base ou endereço inicial.

3 EXEMPLOS DE ALOCAÇÃO DE MEMÓRIA EM SISTEMAS WINDOWS E GNU/LINUX

Agora vamos ver dois simples exemplos de alocação de memória tanto em sistemas Windows e em seguida no sistema Linux.

3.1 Alocação de Memória em Sistemas Windows

```
#include <windows.h>
#include <stdio.h>

int main()
{
    char* my_string = "FIAP - Memory Management";

    LPVOID pAddress = VirtualAlloc(
        NULL,                               // lpAddress
        sizeof(my_string),                  // dwSize
        (MEM_COMMIT | MEM_RESERVE),         // flAllocationType
        PAGE_READWRITE                      // flProtect
    );
    printf("New Address: 0x%p \n", pAddress);
    printf("Len: %ld \n", strlen(my_string));
    printf("Sizeof: %lu\n", sizeof(my_string));

    WriteProcessMemory(
        GetCurrentProcess(),                 // hProcess
        pAddress,                             // lpBaseAddress
        my_string,                           // lpBuffer
        strlen(my_string),                    // nSize
        NULL                                 // *lpNumberOfBytesWritten
    );

    if (VirtualFree(pAddress, 0, MEM_RELEASE))
    {
        printf("Memory clean\n");
    }

    return EXIT_SUCCESS;
}
```

Código-fonte 2 – Um simples exemplo de alocação de memória em sistemas Windows

Fonte: Elaborado pelo autor (2023)

Neste nosso exemplo de alocação de memória, criamos um simples programa que faz o uso de algumas Win32 APIs, tendo como objetivo alocar um espaço de memória virtual em seu próprio processo de ambiente do tamanho de nossa frase (*FIAP – Memory Management*) contendo 24 caracteres.

Aqui vale ressaltar que sempre que trabalhamos com strings (cadeia de caracteres) devemos considerar os caracteres de espaços e também levar em consideração o caractere NULL (também conhecido como nullbyte) \0. Esse caractere tem como objetivo informar ao programa o final de uma string. Então, o correto seria dizer que temos **25 caracteres**.

Então OK, vamos agora entender cada parte do nosso código em detalhes!

```
char* my_string = "FIAP - Memory Management";

LPVOID pAddress = VirtualAlloc(
    NULL,                // lpAddress
    sizeof(my_string),    // dwSize
    (MEM_COMMIT | MEM_RESERVE), // flAllocationType
    PAGE_READWRITE        // flProtect
);
printf("New Address: 0x%p \n", pAddress);
printf("Len: %ld \n", strlen(my_string));
printf("Sizeof: %lu\n", sizeof(my_string));
```

Figura 16 - Alocando memória com a função VirtualAlloc
Fonte: Elaborada pelo autor (2023)

Nessa primeira etapa, criamos um ponteiro `char*` chamado `my_string` que armazenará nossa string de 25 caracteres (*FIAP – Memory Management* + \0). Em seguida declaramos outra variável do tipo `LPVOID` com o nome de `pAddress`, logo em seguida fazemos o uso da função `VirtualAlloc()`, **essa tem como objetivo reservar, confirmar, ou alterar o estado de uma página no espaço de endereçamento virtual do nosso programa**. A memória que é alocada por esta função é inicializada automaticamente vazia. Agora vamos entender alguns dos argumentos passados para essa função:

1. **lpAddress**: Esse parâmetro recebe o endereço de memória que de onde será alocado o espaço. No nosso caso esse valor passado é `NULL`, ou seja, a função será responsável por escolher um endereço onde será alocado.

2. **dwSize**: Recebe o tamanho (em bytes) da região de memória a ser alocado. Nosso caso **25 bytes**.
3. **flAllocationType**: O tipo de alocação que será utilizado. Aqui utilizamos as constantes `MEM_COMMIT` ou `MEM_RESERVE`. Já explicadas anteriormente.
4. **flProtect**: A proteção da região de memória. No nosso caso utilizamos a contrante `PAGE_READWRITE`, pois não iremos executar nada nesse espaço de memória.

Depois apenas é imprimidos os resultados na tela usando a função `printf()`.

```
WriteProcessMemory(  
    GetCurrentProcess(),    // hProcess  
    pAddress,               // lpBaseAddress  
    my_string,              // lpBuffer  
    strlen(my_string),      // nSize  
    NULL,                  // *lpNumberOfBytesWritten  
);
```

Figura 17 - Usando a função `WriteProcessMemory`
Fonte: Elaborada pelo autor (2023)

Nossa segunda etapa foi escrever nossa string no buffer alocado anteriormente utilizando a função `WriteProcessMemory()`. Uma das vantagens das APIs do Windows são seus nomes bem descritivos das funções.

1. **hProcess**: Esse parâmetro recebe um handle do processo ao qual irá ser realizado a escrita na memória. No nosso caso usamos outra função `GetCurrentProcess()` que retornará o handle do nosso próprio programa. No começo o tipo de dado `HANDLE` pode ser confuso. Mas basicamente, **um handle são objetos que estão abertos ou serão criados pelo sistema operacional, como por exemplo, arquivos ou um processo**.
2. **lpBaseAddress**: Um ponteiro para o endereço base que foi criado e alocada anteriormente.
3. **lpBuffer**: Passamos nossa variável que contém a nossa string, que será escrita nesse espaço de memória que alocamos.

4. **nSize**: Utilizamos a função `strlen()`, que irá nos retornar o tamanho da nossa string sem o nullbyte `\0`. Ou seja, apenas **24 caracteres**.
5. ***lpNumberOfBytesWritten**: Esse parâmetro recebe um ponteiro para uma variável que contém o número de bytes transferidos para o processo especificada. No nosso caso passamos o valor `NULL`, então esse parâmetro será ignorado.

Na etapa final, realizamos a liberação (ou deslocamento) do espaço de memória que alocamos anteriormente, já que não iremos utilizar mais. Para isso utilizamos a função `VirtualFree()`.

1. **lpAddress**: Um ponteiro para o endereço base da região a ser liberada.
2. **dwSize**: O tamanho da região a ser liberado (em bytes).
3. **dwFreeType**: O tipo de liberação a ser utilizada. No nosso caso usamos a contante `MEM_RELEASE`, essa tem como objetivo liberar a região específica de página ou espaço reservado. Após esta operação, as páginas ficam no estado livre.

Nas figuras abaixo, podemos ver cada etapa que o programa executa na memória.

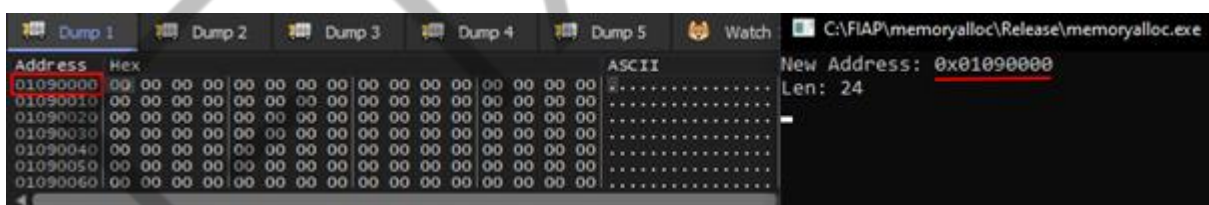


Figura 18 - Alocando espaço de memória no endereço 0x1090000

Fonte: Elaborada pelo autor (2023)

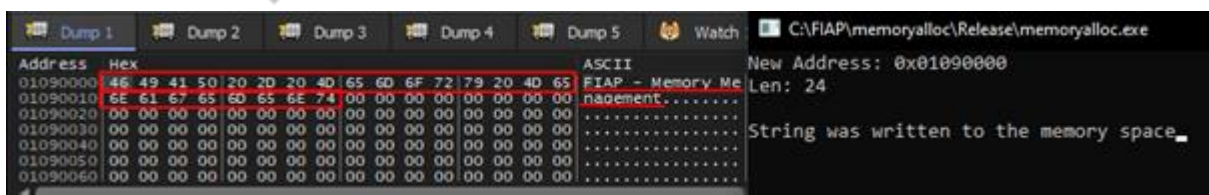


Figura 19 - Escrevendo no espaço de memória alocado anteriormente

Fonte: Elaborada pelo autor (2023)

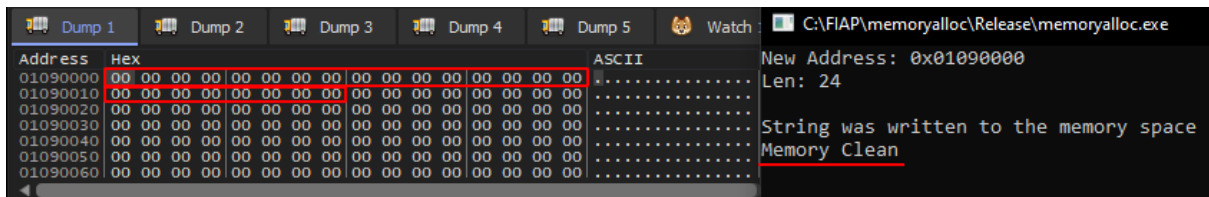


Figura 20 - Liberação do espaço de memória.

Fonte: Elaborada pelo autor (2023)

3.2 Alocação de memória em sistema GNU/Linux

```
#include <stdio.h>
#include <sys/mman.h>
#include <string.h>

#define SIZE 1024

int main()
{
    void* ptr;

    ptr = mmap(
        NULL,
        SIZE,
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANONYMOUS,
        -1,
        0,
        // void *address
        // size_t length
        // int protect
        // int flags
        // int filedes
        // off_t offset
    );
    printf("New Address: %p \n", ptr);
    if (ptr == MAP_FAILED)
    {
        perror("Error mmap\n");
        return -1;
    }

    /*
     * char* str
     * const char* format
     */
    sprintf(ptr, "FIAP - Memory Management");
    printf("Len: %d\n", strlen(ptr));
    printf("String was written to the memory space\n");

    /*
     * void* addr
     * size_t len
     */
    if (munmap(ptr, SIZE) == -1)
    {
        perror("Error: munmap\n");
    }
    else { printf("Memory Clean\n"); }

    return 0;
}
```

Código-fonte 3 – Um simples exemplo de alocação de memória em sistemas GNU/Linux

Fonte: Elaborado pelo autor (2023)

Novamente iremos utilizar um simples código em C que irá fazer a alocação de um bloco de memória, só que agora no sistema GNU/Linux.

A forma como o Linux performa o Gerenciamento de Memória é bem diferente de como o Windows gerencia. Algumas diferenças que são perceptíveis podem ser vista na lista abaixo:

- **Gerenciamento de memória compartilhada:** Os sistemas Linux possuem suporte nativo a memória compartilhada. Isso permite que processos compartilhem regiões de memória, o que é útil em cenários de comunicação intraprocessual. O Windows também oferece suporte à memória compartilhada, porém requer o uso de objetos de kernel específicos para realizar essa funcionalidade.
- **Controle de permissões de memória:** O Linux faz o uso do modelo de permissões baseado em usuário e grupos para controlar o acesso à memória, utilizando permissões de leitura, escrita e execução. Já os sistemas da Microsoft fazem o uso de um modelo de controle de acesso baseado em listas de controle de acesso (*Access Control Lists* - ACLs) utilizada para definir permissões de memória.
- **Algoritmos de substituição de página:** Ambos os sistemas operacionais implementam algoritmos de substituição de página para gerenciar a memória virtual. O GNU/Linux faz o uso principalmente do algoritmo *Least Recently Used* (LRU). Enquanto o Windows utiliza o algoritmo *Modified Clock* (também conhecido como *Second-Chance*) em combinação com outros algoritmos.

Agora vamos entender como é feito a alocação de memória com o nosso código.

Na primeira etapa vemos claramente que a nomenclatura das funções é bastante diferente, isso porque nos sistemas Linux não é possui o conceito de “API”, esse conceito é do sistema Windows que tem como objetivo facilitar no desenvolvimento de software.

```
void* ptr;

ptr = mmap(
    NULL,                // void *address
    SIZE,                 // size_t length
    PROT_READ | PROT_WRITE, // int protect
    MAP_PRIVATE | MAP_ANONYMOUS, // int flags
    -1,                  // int filedes
    0                     // off_t offset
);
printf("New Address: %p \n", ptr);
```

Figura 21 - Utilizando a função mmap no Linux para alocar memória
Fonte: Elaborada pelo autor (2023)

Para podemos realizar a alocação de um espaço de memória virtual utilizamos a função `mmap()`, que irá armazenar o espaço desejado pelo nosso programa, retornando o valor para nossa variável `ptr`.

1. **void *address:** Esse parâmetro é bem similar com o que vimos anteriormente na função `VirtualAlloc`. Se caso não for passado num um valor, a própria função escolherá um novo endereço.
2. **size_t length:** Esse parâmetro recebe uma constante que definimos logo no começo do código com o valor de 1024.
3. **int protect:** Este argumento é usado para controlar que tipo de acesso é permitido. No nosso caso utilizamos as constantes `PROT_READ` ou `PROT_WRITE` passando permissão de escrita e leitura.
4. **int flag:** Este argumento é usado para controlar a natureza do mapa, possibilitando o uso de 4 valores. No nosso caso utilizamos apenas duas constantes, `MAP_PRIVATE` que irá mapear esse endereço de memória de uma forma que nem outro processo poderá ver, já a constante `MAP_ANONYMOUS` (ou `MAP_ANON`), irá criar um mapeamento anônimo. Isso significa que esse mapeamento não poderá ser acessado por qualquer arquivo.
5. **int filedes:** Esse parâmetro recebe o descritor de arquivo que deve ser mapeado. No nosso caso passamos o valor **-1**, que significa que não há manipulação de arquivos relacionado a alocação de memória.
6. **off_t offset:** Especifica que o deslocamento dentro do arquivo relacionado à alocação de memória é zero.

Em seguida imprimimos o novo endereço de memória alocado na tela e realizamos uma validação para identificar se nossa memória foi alocada de forma correta.

Na segunda etapa do nosso programa, fazemos o uso da função `sprintf()`. Essa função é diferente da `printf()` que imprime um resultado na tela. Em vez de ser impresso, o conteúdo é armazenado como uma string no buffer apontando pelo parâmetro `ptr`.

```

/*
void* addr
size_t len
*/
if (munmap(ptr, SIZE) == -1)
{
    perror("Error: munmap\n");
}
else { printf("Memory Clean\n"); }

```

Figura 22 - Exemplo de escrita de memória em sistema GNU/Linux
Fonte: Elaborada pelo autor (2023)

No final após realizarmos as devidas manipulações na memória, utilizamos a função `munmap()`, que tem como objetivo liberar o espaço de memória virtual, dessa forma a região de memória que foi alocada entrara no estado livre. Os argumentos passados para essa função foram o endereço da memória, e o tamanho do espaço de memória a ser desalocado.

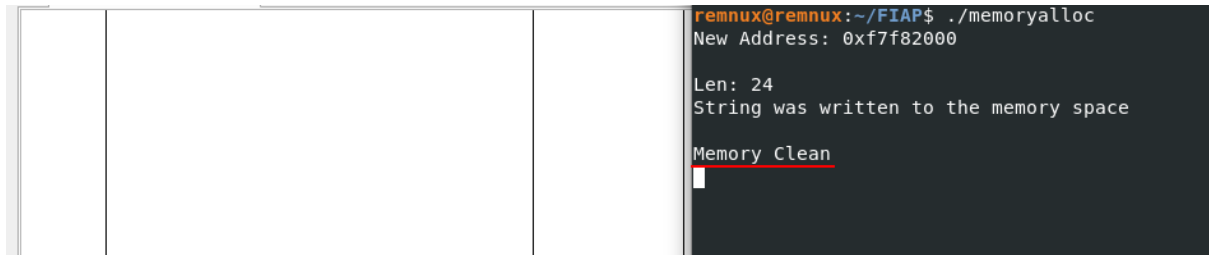
Abaixo podemos ver como é feito o gerenciamento de memória em tempo de execução no sistema GNU/Linux.

The image shows a memory dump on the left with addresses from f7f8:2000 to f7f8:20c0. Each address is followed by a 16-byte hex dump. On the right, a terminal window shows the command `./memoryalloc` being executed, resulting in the output: `New Address: 0xf7f82000`.

Figura 23 - Alocando espaço de memória no endereço 0xF7F82000
Fonte: Elaborada pelo autor (2023)

The image shows a memory dump on the left with addresses from f7f8:2000 to f7f8:20c0. The first few bytes of the first row (f7f8:2000) are highlighted with a red box and contain the hex values 46, 49, 41, 50, 20, 2d, 20, 4d, 65, 6d, 6f, 72, 79, 20, 4d, 61. On the right, a terminal window shows the command `./memoryalloc` being executed, resulting in the output: `New Address: 0xf7f82000`, `Len: 24`, and `String was written to the memory space`.

Figura 24 - Escrevendo no espaço de memória alocando anteriormente
Fonte: Elaborada pelo autor (2023)

A terminal window with a dark background. The prompt is 'remnux@remnux:~/FIAP\$'. The command './memoryalloc' has been executed. The output shows 'New Address: 0xf7f82000', 'Len: 24', and 'String was written to the memory space'. Below this, the text 'Memory Clean' is displayed with a red underline. A small white cursor is visible at the end of the line.

```
remnux@remnux:~/FIAP$ ./memoryalloc
New Address: 0xf7f82000

Len: 24
String was written to the memory space

Memory Clean
```

Figura 25 - Liberação de espaço de memória
Fonte: Elaborada pelo autor (2023)

Com isso conseguimos obter um conhecimento básico de como é feito a alocação e a liberação de memória nos dois sistemas operacionais mais utilizados no mundo. No próximo tópico falaremos sobre ataques e vulnerabilidade de memória.

4 ATAQUES EM MEMÓRIA

No mundo da segurança cibernética, as vulnerabilidades em memória são consideradas uma das ameaças mais sérias e explorados por invasores. Essas vulnerabilidades existem tanto em sistemas GNU/Linux como em sistemas Windows. As vulnerabilidades em memória são falhas presentes nos sistemas operacionais ou aplicações que foram desenvolvidas incorretamente.

Essas falhas podem ser exploradas por meio de ataques de memória, que se aproveitam de erros de programação ou configuração inadequada permitindo acesso privilegiado, acesso remoto ou até mesmo a disponibilidade do sistema.

4.1 Fork Bomb

O *Fork Bomb* é um ataque de negação de serviço (*Denial of Service – DoS*) em sistemas operacionais. Esse ataque pode ocorrer de uma forma simples utilizando uma sequência de caracteres como `( : () { : | : & } ; : )`. Sim, essa sequência esquisita de caracteres pode derrubar um sistema Linux caso executado no terminal. Então vamos quebrá-lo em partes para entender melhor.

1. O caractere de dois pontos (:) é um comando no-op válido no Linux. Não faz nada e retorna verdadeiro.

- Esse comando consiste em uma definição de função `() { ... }` seguindo de um ponto e vírgula `( ; )` e dois pontos no final `(:)`. Definindo a seguinte função:
`: () { ... }`
- Vemos que o nome da função é os dois pontos `(:)`. Dentro da função, temos uma chamada recursiva para a função `(:)` seguido por um símbolo de pipe `( | )` e por outro dois pontos. O símbolo `( & )` no final executa a função em background, possibilitando múltiplas instancias serem executadas simultaneamente.

Então, o que acontece é que cada instância da função `:` gera mais duas instâncias dela mesma em segundo plano, criando um crescimento exponencial no número de processos. Isso continua até que o sistema fique sobrecarregado e sem resposta, levando potencialmente a uma falha do sistema ou à necessidade de uma reinicialização.

Vale ressaltar que este comando nunca deve ser executado, já que pode causar sérios danos ao sistema e atrapalhar seu funcionamento normal.

Em um ambiente de laboratório vamos executar esse programa. Para fins demonstrativos eu alterei a quantidade de memória da minha máquina virtual para **2 GB**.

Como podemos ver na figura abaixo, nosso sistema está fazendo o uso de cerca de **873 MB** de memória.

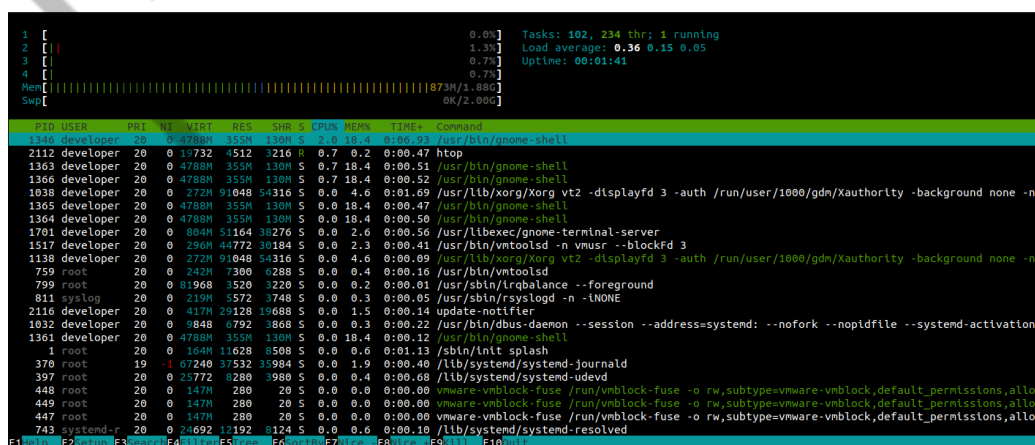


Figura 26 - Uso de memória do nosso sistema-alvo

Fonte: Elaborada pelo autor (2023)

Após realizarmos a execução do comando veremos que o sistema entrará em colapso deixando nossa máquina inutilizável.

```

1 [ 0.0% Tasks: 103, 233 thr; 1 running
2 [ 0.7% Load average: 0.46 0.16 0.06
3 [ 0.7% Uptime: 00:00:56
4 [ 0.7%
Mem[ 861M/1.88G
Swp[ 0K/2.00G

PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+
1025 developer 20 0 277M 91420 56284 S 0.7 4.6 0:02.85
1716 developer 20 0 19616 4240 3176 R 0.7 0.2 0:00.28
1528 developer 20 0 294M 41456 29568 S 0.7 2.1 0:00.32
1318 developer 20 0 4532M 355M 131M S 0.0 18.4 0:09.73
1687 developer 20 0 808M 52568 38080 S 0.0 2.7 0:01.13
1112 developer 20 0 277M 91420 56284 S 0.0 4.6 0:00.36
1348 developer 20 0 4532M 355M 131M S 0.0 18.4 0:00.94
1347 developer 20 0 4532M 355M 131M S 0.0 18.4 0:00.96
1349 developer 20 0 4532M 355M 131M S 0.0 18.4 0:01.02
1346 developer 20 0 4532M 355M 131M S 0.0 18.4 0:01.02
987 developer 20 0 26128 16680 8028 S 0.0 0.5 0:00.24
1019 developer 20 0 242M 6912 5928 S 0.0 0.3 0:00.03
1357 developer 20 0 4532M 355M 131M S 0.0 18.4 0:00.07
796 syslog 20 0 219M 5624 3796 S 0.0 0.3 0:00.05
798 root 20 0 238M 6588 5908 S 0.0 0.3 0:00.02
1366 developer 20 0 277M 31544 18004 S 0.0 1.6 0:00.83
1473 developer 20 0 346M 28508 18548 S 0.0 1.4 0:00.17
748 root 20 0 242M 7296 6272 S 0.0 0.4 0:00.11
1 root 20 0 164M 11388 8244 S 0.0 0.6 0:00.97
776 avahi 20 0 8532 3444 3120 S 0.0 0.2 0:00.03
369 root 19 -1 75416 41396 39852 S 0.0 2.1 0:00.37
783 messageb 20 0 8800 8820 3920 S 0.0 0.3 0:00.21

fork_bomb() { : | : & } :;
[1] 1731
fork_bomb() { : | : & } :;
[1] 1731

```

Figura 27 - Execução do comando de fork bomb com o PID de 1731

Fonte: Elaborada pelo autor (2023)

Assim que é executado esse comando nossa máquina congela, impossibilitando o uso dela.

```

1 [ 58.9% Tasks: 125, 233 thr; 2 running
2 [ 59.9% Load average: 1361.56 381.66 130.29
3 [ 59.2% Uptime: 00:01:43
4 [ 59.6%
Mem[ 1.35G/1.88G
Swp[ 1.51M/2.00G

PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
4754 developer 20 0 0 0 0 0 18.5 0.0 0:00.28 bash
987 developer 20 0 26128 16680 8028 S 15.2 0.5 0:00.47 /lib/systemd/systemd --user
2708 developer 20 0 19384 7984 1512 S 11.2 0.2 0:00.17 bash
4913 developer 20 0 0 0 0 0 11.2 0.0 0:00.17 bash
4809 developer 20 0 0 0 0 0 7.9 0.0 0:00.12 bash
4889 developer 20 0 0 0 0 0 7.9 0.0 0:00.12 bash
3622 developer 20 0 0 0 0 0 7.3 0.0 0:00.11 bash
4711 developer 20 0 0 0 0 0 7.3 0.0 0:00.11 bash
4780 developer 20 0 19384 1360 0 0 7.3 0.1 0:00.11 bash
4940 developer 20 0 0 0 0 0 7.3 0.0 0:00.11 bash
4637 developer 20 0 0 0 0 0 6.6 0.0 0:00.10 bash
4805 developer 20 0 0 0 0 0 6.6 0.0 0:00.10 bash
5029 developer 20 0 0 0 0 0 6.6 0.0 0:00.10 bash
5112 developer 20 0 19384 1364 0 0 6.6 0.1 0:00.10 bash
4014 developer 20 0 0 0 0 0 6.0 0.0 0:00.09 bash
4597 developer 20 0 0 0 0 0 6.0 0.0 0:00.09 bash
4936 developer 20 0 0 0 0 0 6.0 0.0 0:00.09 bash
3840 developer 20 0 0 0 0 0 5.3 0.0 0:00.08 bash
4268 developer 20 0 0 0 0 0 5.3 0.0 0:00.08 bash
4465 developer 20 0 0 0 0 0 5.3 0.0 0:00.08 bash
4729 developer 20 0 0 0 0 0 5.3 0.0 0:00.08 bash
4742 developer 20 0 0 0 0 0 5.3 0.0 0:00.08 bash
5208 developer 20 0 19384 7032 672 R 5.3 0.1 0:00.08 bash
4263 developer 20 0 0 0 0 0 4.6 0.0 0:00.07 bash

bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable
bash: fork: retry: Resource temporarily unavailable

```

Figura 28 - Recursos ao extremo

Fonte: Elaborada pelo autor (2023)

Vemos algumas mensagens repetidas no terminal nos informando que não há recurso temporariamente disponível. Vemos também um aumento drástico do

consumo de memória e de CPU. Em seguida ficou impossível utilizar a máquina necessitando reiniciá-la.

Para mais básica que pode ser utilizada para mitigar tal tipo de ataque em seus sistemas Linux, é o uso do comando `ulimit -u` esse comando prove o número máximo que um usuário pode criar em utilizando uma shell. Esse comando também permite configurar o controle sobre os recursos disponíveis para o shell e seus processos criados.

5 PROTEÇÕES DE MEMÓRIA

Agora que temos um bom entendimento sobre o gerenciamento de memória, vamos explorar diferentes técnicas de proteção de memória implementadas ao longo do tempo em sistemas operacionais. Ao compreendemos esses mecanismos de proteção, estaremos mais preparados para enfrentar os desafios de segurança.

5.1 *Garbage Collection*

Como mencionado anteriormente linguagem como C/C++ não possuem coleta de lixo. Deixando a responsabilidade do desenvolvedor alocar e liberar os recursos de memória de forma manual. O *Garbage Collection* é um processo automatizado e essencial na gestão de memória em linguagens de programação mais modernas. A responsabilidade por lidar com a alocação e a liberação de memória passar a ser responsabilidade da linguagem em si.

O coletor de lixo realiza um escaneamento da memória e verifica quais não estão sendo utilizadas pelo programa e assim identificando quais áreas de memória não estão sendo mais referenciadas. Essas áreas de memória não referenciadas são consideradas “lixo” e são elegíveis para serem liberadas.

Linguagens que fazem o uso essa técnica são:

- Rust
- Java
- Python

- C#

Vamos ver um exemplo de código que não faz o gerenciamento de memória adequado deixando “lixo” na memória.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char* STRING = "This is Garbage!";

    void* my_string = malloc(strlen(STRING) + 1);
    printf("My variable address: 0x%p", my_string);

    strcpy(my_string, STRING);
    printf("String was copied to our address.\nNow we are exiting...\n");

    return 0;
}
```

Código-fonte 4 – Exemplo de programa que não faz a liberação de memória
Fonte: Elaborado pelo autor (2023)

Neste simples programa, usamos a função `malloc()` para alocar espaço de memória de forma dinâmica para a string *This is Garbage!*, em seguida, copiamos essa string para o espaço de memória alocando utilizando a `strcpy()`. No entanto, depois de usar a string, esse programa não faz o tratamento adequado para liberar o espaço de memória. Isso significa que o espaço na memória foi desalocado, mas o conteúdo ainda está presente na memória, resultando em lixo de memória como demonstrado na Figura 29 abaixo

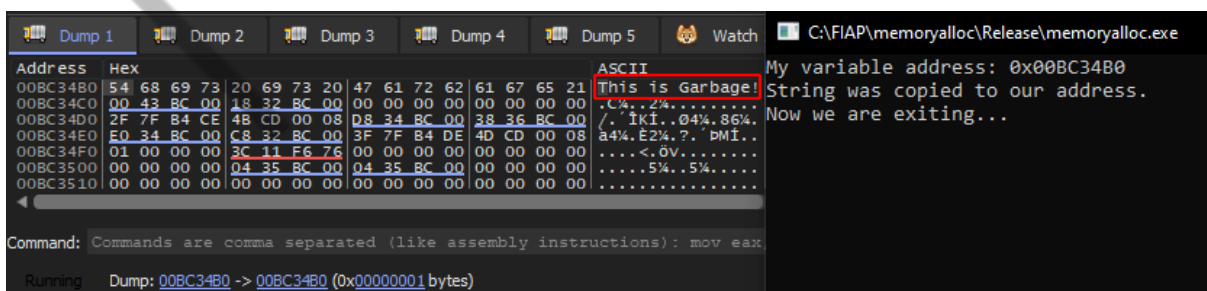


Figura 29 - Lixo deixado na memória
Fonte: Elaborada pelo autor (2023)

Para resolver esse problema basta fazer o uso da função `free()` quando não for mais necessário a utilização do espaço alocado.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char* STRING = "This is Garbage!";

    void* my_string = malloc(strlen(STRING) + 1);
    printf("My variable address: 0x%p", my_string);
    strcpy(my_string, STRING);

    free(my_string);
    printf("String was copied to our address.\nOur memory was deallocated\nNow
we are exiting...\n");

    return 0;
}
```

Código-fonte 5 – Exemplo de código realizando o tratamento adequado de memória
Fonte: Elaborado pelo autor (2023)

Nas figuras “Alocação de memória” e “Liberação de memória” podemos ver os resultados do tratamento adequado de memória.

Address	Hex	ASCII
00EF3480	54 68 69 73 20 69 73 20 47 61 72 62 61 67 65 21	This is Garbage!
00EF34C0	00 43 EF 00 18 32 EF 00 00 00 00 00 00 00 00	.C1..21.....
00EF34D0	16 09 53 0A 3D 2C 00 08 08 34 EF 00 38 36 EF 00	..S.=...04i.86i.
00EF34E0	E0 34 EF 00 C8 32 EF 00 06 09 53 1A 38 2C 00 08	à4i.E2i...S.;...
00EF34F0	01 00 00 00 3C 11 F6 76 00 00 00 00 00 00 00	<.0v.....
00EF3500	00 00 00 00 04 35 EF 00 04 35 EF 00 00 00 00	5i..5i.....
00EF3510	00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Figura 30 - Alocação de memória
Fonte: Elaborada pelo autor (2023)

Address	Hex	ASCII
00EF3480	54 68 69 73 20 69 73 20 47 61 72 62 61 67 65 21	This is Garbage!
00EF34C0	00 43 EF 00 18 32 EF 00 00 00 00 00 00 00 00	.C1..21.....
00EF34D0	16 09 53 0A 3D 2C 00 08 08 34 EF 00 38 36 EF 00	..S.=...04i.86i.
00EF34E0	E0 34 EF 00 C8 32 EF 00 06 09 53 1A 38 2C 00 08	à4i.E2i...S.;...
00EF34F0	01 00 00 00 3C 11 F6 76 00 00 00 00 00 00 00	<.0v.....
00EF3500	00 00 00 00 04 35 EF 00 04 35 EF 00 00 00 00	5i..5i.....
00EF3510	00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Figura 31 - Liberação de Memória
Fonte: Elaborada pelo autor (2023)

5.2 Data Execution Prevention (DEP)

A prevenção de execução de dados (*Data Execution Prevention* - DEP) é uma técnica de proteção que visa evitar a execução de código que se encontra em áreas de memória designadas para armazenamentos apenas de dados. O DEP marca essa área como não executáveis, impedindo que instruções de código sejam executadas nelas. Dessa forma, vulnerabilidades de segurança que exploram a execução de código em áreas de dados podem ser mitigadas. Porém, nada é 100% seguro e

existem formas de burlar esse mecanismo de segurança, uma delas é a programação orientada ao retorno (*Return Oriented Programming - ROP*), no entanto isso está fora do nosso escopo.

5.3 Address Space Layout Randomization (ASLR)

A aleatorização de espaço de endereço (*Address Space Layout Randomization - ASLR*) é uma técnica de segurança amplamente utilizada para proteger sistemas operacionais e aplicações contra os ataques que exploram a memória. O objetivo principal do ASLR é dificultar a previsibilidade da localização de funções e dados armazenados na memória, tornando mais difícil para os atacantes explorarem essas vulnerabilidades.

Quando o ASLR está ativado, o sistema operacional randomiza a posição na memória em que são carregados os diferentes componentes de um programa, como bibliotecas, pilhas e segmentos de código. Isso significa que cada vez que uma aplicação é executada seu endereço de memória são alocados em diferentes posições

CONCLUSÃO

Com esse capítulo, obtemos um entendimento bastante abrangente sobre o Gerenciamento de Memória, exploramos uma variedade de tópicos relacionados a esse fascinante assunto e essencial para o sistema operacional. Vimos diferentes tipos de esquema de memória e sua evolução ao longo do tempo, exploramos os conceitos de memória virtual. Falamos sobre vulnerabilidades que podem comprometer a disponibilidade de um sistema, e as mitigações que forma empregadas para solucionar tais problemas. Vimos alguns exemplos de alocação de memória em sistemas Windows e GNU/Linux. Com esse conhecimento podemos apreciar a complexidade a importância do gerenciamento de memória na criação de sistemas e softwares confiáveis, eficientes e seguros.

REFERÊNCIAS

FULBER-GARCIA, Vinicius. Memory Management. Baeldung, 8 jun. 2023. Disponível em: <<https://www.baeldung.com/cs/os-memory-management>>. Acesso em: 19 jun. 2023.

MICROSOFT. Sobre o gerenciamento de memória. 12 jun. 2023. Disponível em: <<https://learn.microsoft.com/pt-br/windows/win32/memory/about-memory-management>>. Acesso em: 19 jun. 2023.

PREPINSTA. First fit in C. s.d. Disponível em: <<https://prepinsta.com/operating-systems/page-replacement-algorithms/first-fit/first-fit-in-c/>>. Acesso em: 19 jun. 2023.

YOSIFOVICH, Pavel et al. **Windows Internals 7**. Washington: Microsoft Press, 2017.